

Viva Preparation — Ziplink URL Shortener

1. Project Overview Questions

Q: What is this project about?

A: A URL shortener dashboard built with React. Users register/login, paste a long URL, and get a short link via Shlink (a self-hosted URL shortening API). They can manage their links — search, sort, paginate, edit, delete, copy to clipboard, and generate QR codes.

Q: What is the tech stack?

A: React 19 for UI, React Router DOM 7 for routing, Bootstrap 5 for styling, Zustand 5 for state management, bcryptjs for password hashing, Create React App for build tooling, and Docker for hosting the Shlink API backend. All user data is stored in browser localStorage.

Q: Why no backend database?

A: This is a frontend demo project. Storing data in localStorage keeps it self-contained — no server setup needed beyond the Shlink Docker container. It's not production-ready but demonstrates all the core features of a URL shortener.

2. Architecture Questions

Q: Explain the app's architecture.

A: It's a single-page React app with three layers: 1. **UI Layer** — Components (Navbar, LinkTable, modals, etc.) and pages (Login, Register, Dashboard) 2. **State Layer** — Two Zustand stores: authStore (user session) and linksStore (CRUD operations) 3. **Service Layer** — auth.js (authentication with bcrypt), shlink.js (Shlink API client), utils.js (helpers)

The Shlink API runs in a Docker container separately. The app talks to it via HTTP fetch requests.

Q: What is the role of the Dashboard component?

A: Dashboard.js is the orchestrator (~100 lines). It holds all UI state (url, search, sort, page, modal flags), wires up store actions, handles the URL shortening logic, and renders all child components. It uses useCallback for event handlers and useMemo for the filtered/sorted link list.

Q: Where is the routing defined?

A: In App.js. Three routes: `/login`, `/register`, `/dashboard`. The dashboard is wrapped in a `ProtectedRoute` component that checks `useAuthStore` — if no user is logged in, it redirects to `/login`. Any other path redirects to `/dashboard`.

3. Authentication Questions

Q: How does registration work?

A: Register.js calls `registerUser(email, password)` from auth.js. This reads `localStorage['shlink_users']`, checks for duplicate emails, hashes the password with `bcrypt.hash(password, 10)`, pushes the new user object, and saves back. Returns `{ ok: true }` on success or `{ ok: false, error: 'Email already registered' }` on failure.

Q: How does login work?

A: Login.js calls `loginUser(email, password)`. It finds the user in `localStorage`, then checks the stored password: - If it starts with `$2` (bcrypt format), it uses `bcrypt.compare(password, hash)` - If it's old plaintext, it does a direct string comparison and on success, upgrades to bcrypt

On success, it writes the session to `localStorage` and calls `authStore.login(email)`.

Q: How is the session maintained?

A: Zustand's `persist` middleware automatically syncs `authStore` state to `localStorage['shlink_session']`. When the app loads, Zustand hydrates the store from `localStorage`. Any component can read `useAuthStore(s => s.user)` — it's reactive, so the UI updates instantly on login/logout without a page refresh.

Q: How does logout work?

A: Dashboard calls `authStore.logout()`, which sets `user: null` in Zustand. The `persist` middleware updates `localStorage`. Then `navigate('/login')` redirects the user. `ProtectedRoute` re-renders and redirects because `user` is now null.

Q: Why bcryptjs instead of plaintext?

A: bcrypt is a one-way hash function with built-in salt (10 rounds here). Even if `localStorage` is compromised, passwords can't be recovered. It's the industry standard for password storage.

4. URL Shortening Questions

Q: Walk through the URL shortening flow.

A: 1. User pastes a URL and clicks “+ Add Link” 2. Dashboard checks the 5-link limit (skipped for the demo Pro user `prodemo@gmail.com`) 3. Shows a loading spinner on the button 4. Calls `shlink.js` → `shortenUrl(url)` which sends `POST /rest/v3/short-urls` to Shlink with `X-API-Key` header 5. Shlink returns `{ shortUrl, longUrl, title }` — it auto-fetches the page title 6. The title is cleaned via `cleanTitle()` (strips HTML tags, collapses whitespace) 7. The short URL is normalized via `normalizeShortUrl()` to ensure consistent base 8. An `EditLinkModal` opens in “preview” mode — user can edit title/URL 9. On confirm, `addLink()` prepends to the store and auto-saves to `localStorage` 10. The new link appears at the top of the table

Q: What happens if the Shlink API fails?

A: The fetch throws an error, which is caught in the try/catch block. Instead of `alert()`, we now set an `error` state which renders a dismissible red Bootstrap alert banner above the form with the error message.

Q: What is the 5-link limit and how does it work?

A: Non-Pro users (any email except `prodemo@gmail.com`) can only have 5 links. The check is `links.length >= 5` in `handleShorten()`. When hit, instead of shortening, it shows the `ProModal` upsell — a fake “Subscribe Now — \$4.99/mo” dialog (UI only, no payment integration).

5. State Management Questions

Q: Why Zustand instead of Redux or Context?

A: Zustand is ~1KB, has zero boilerplate, doesn’t need a Provider wrapper, and has built-in `persist` middleware for `localStorage` sync. Redux is overkill for this app’s scale. Context would work but causes re-render issues and doesn’t have `persist`.

Q: How is the links store different from the auth store?

A: `authStore` uses Zustand’s `persist` middleware (auto-syncs to `localStorage`). `linksStore` uses manual `localStorage` calls via `saveLinks()` because the storage key is dynamic per user email — `shlink_links_{email}`. The `persist` middleware doesn’t support dynamic keys easily.

Q: How does per-user link isolation work?

A: Each user's links are stored under a unique key: `shlink_links_{email}`. When Dashboard mounts, `loadLinks(user.email)` reads the correct key. This way, user A's links are completely separate from user B's links in `localStorage`.

6. Component Questions

Q: Explain the ModalWrapper component.

A: ModalWrapper is a reusable shell that renders Bootstrap's modal backdrop and dialog structure. It handles: - Escape key to close (keyboard event listener in `useEffect`) - Backdrop click to close (`onClick` on the backdrop div) - Stop propagation on the dialog content so clicking inside doesn't close

All modals (`QrModal`, `EditLinkModal`, `ProModal`, `ConfirmModal`) use ModalWrapper, sharing this behavior.

Q: How does the QR code display work?

A: Two places: 1. **Sidebar** — clicking a table row sets `selectedLink`. The sidebar shows a 150×150 QR image from `api.qrserver.com` with a line-spinner animation while loading 2. **Modal** — clicking the QR button in the table opens `QrModal` with a 200×200 image and the same line-spinner

Both use `onLoad` on the `<img>` tag to detect when the image is ready and hide the loading indicator.

Q: How does the copy-to-clipboard feature work?

A: Uses the browser Clipboard API: `navigator.clipboard.writeText(shortUrl)`. On click, the button text changes to "Copied!" for 1.5 seconds using a `copiedId` state variable and `setTimeout`.

Q: How does delete with confirmation work?

A: Clicking the trash icon sets `deleteTarget` to the link object. This renders `ConfirmModal` with the link's title. The modal has Cancel and Delete buttons. Delete calls `deleteLink(deleteTarget.short)` which filters it out of the store and auto-saves. Cancel sets `deleteTarget` to null, dismissing the modal.

7. Search, Sort, Pagination Questions

Q: How is search implemented?

A: A controlled input with `search` state. The `filtered` array is computed via `useMemo` — it filters links where `title` or `long` includes the search query (case-insensitive). When search changes, `useEffect` resets page to 1. The memo prevents unnecessary re-filtering on unrelated state changes.

Q: How does sorting work?

A: A dropdown with “Descending” (newest first) and “Ascending” (oldest first) options. The sort is applied in the same `useMemo` using `Date` comparison on the `date` field.

Q: How does pagination work?

A: `PER_PAGE = 3`. The visible links are `filtered.slice((page - 1) * PER_PAGE, page * PER_PAGE)`. The Pagination component shows Previous/Next buttons, page numbers, and a “Show All” toggle. When “Show All” is active, all filtered links are shown without slicing.

8. External API Questions

Q: What external APIs does the app use?

A: Three: 1. **Shlink** (self-hosted) — `POST /rest/v3/short-urls` to create short URLs 2. **Google Favicons** — `//www.google.com/s2/favicons?domain=X&sz=16` for link row favicons 3. **QR Server** — `//api.qrserver.com/v1/create-qr-code` for QR code images

Q: How is the Shlink API key handled?

A: It’s stored in a `.env` file as `REACT_APP_API_KEY` and read via `process.env.REACT_APP_API_KEY` in `shlink.js`. The `.env` file is in `.gitignore` to prevent committing secrets. There’s a fallback for development (empty string), relying on the env var being set at runtime.

9. Security Questions

Q: What security measures are in place?

A: - Passwords hashed with `bcrypt` (10 salt rounds) — not stored in plaintext - Old plaintext passwords auto-migrated to `bcrypt` on next login - API key moved to `.env` file (not hardcoded in source anymore) - `.env` added to `.gitignore`

Q: What security issues remain?

A: - localStorage is accessible to any JavaScript on the same origin (XSS vulnerability) - No session expiry — user stays logged in until explicit logout - No HTTPS between frontend and Shlink (API key sent in cleartext on LAN) - Per-user isolation is naming convention only, not enforced

10. Testing Questions

Q: What tests are written?

A: 71 tests across 8 suites: - **utils.test.js** — 15 tests for cleanTitle, extractDomain, normalizeShortUrl, constants - **auth.test.js** — 12 tests for register, login (bcrypt + plaintext migration), logout, session, linksKey - **authStore.test.js** — 5 tests for login/logout and Zustand persist behavior - **linksStore.test.js** — 13 tests for loadLinks, addLink, updateLink, deleteLink, saveLinks, URL normalization - **ConfirmModal.test.js** — 7 tests for render, buttons, Escape key, backdrop click, stopPropagation - **EditLinkModal.test.js** — 8 tests for edit/preview mode, pre-filled values, save with updates - **ProModal.test.js** — 4 tests for render, close button, Escape key - **QrModal.test.js** — 6 tests for render, line spinner, close button, null guard, image URL

Q: What testing library is used?

A: React Testing Library with Jest (built into Create React App). Tests use `render`, `screen`, `fireEvent` from `@testing-library/react` and `jest.fn()` for mocks.

11. React Concepts Questions

Q: What hooks are used and where?

A: - `useState` — all component-local state (form inputs, modals, loading, search, sort, page) - `useEffect` — load links on mount, reset pagination on search, Escape key listener in ModalWrapper - `useCallback` — event handlers in Dashboard (handleShorten, handleConfirm, etc.) to prevent re-renders - `useMemo` — filtered/sorted link list to avoid recomputation on every render

Q: Why useCallback and useMemo?

A: `useCallback` prevents child components from re-rendering when the parent re-renders, since the function reference stays the same. `useMemo` caches the filtered/sorted array so it's only recalculated when `links`, `search`, or `sort` actually change — not on every state update (like typing in the search box).

Q: How is conditional rendering used?

A: Extensively: `{condition && <Component />}` for modals (QrModal, ConfirmModal, ProModal), ternary for edit/preview mode, `{links.length === 0 && <EmptyState/>}` for the table, and `{loading ? <Spinner /> : 'Add Link'}` for the submit button.

Q: What is the key prop and why does it matter?

A: React uses the `key` prop to identify elements in a list. We use `key={link.short}` (the actual short URL) instead of array index `key={i}`. This ensures React correctly tracks each row across re-renders, preserving state and avoiding bugs when items are reordered, filtered, or deleted.

12. Common Viva Questions

Q: What was the biggest challenge?

A: Refactoring the monolithic Dashboard into separate components. The original had ~480 lines with everything mixed together. Splitting it required understanding all the data dependencies, extracting reusable patterns (like ModalWrapper), and ensuring state flows correctly between components.

Q: What would you improve?

A: 1. Add a real backend (Express + SQLite) for multi-device support 2. Migrate from CRA to Vite (CRA is deprecated) 3. Add virtual scrolling for large link lists 4. Implement analytics (click tracking via Shlink visits endpoint) 5. Add proper unit tests with higher coverage

Q: How is this different from a production URL shortener?

A: Production shorteners (like Bitly) have: - Server-side databases (not local-Storage) - User authentication with JWT tokens and expiry - HTTPS everywhere - Click analytics and tracking - Custom short domains - Rate limiting - Multi-device sync - Proper deployment with CI/CD

This project demonstrates the frontend concepts but uses localStorage as a simplification.

Q: What is Zustand's persist middleware?

A: It automatically saves the store state to a storage backend (localStorage by default) on every change. On app load, it hydrates the store from storage. The storage key is configurable — we use `shlink_session` for auth. This means session survives page refreshes and browser restarts without any manual `getItem/setItem` calls.

Q: How do you handle errors in API calls?

A: Try/catch/finally blocks. The catch sets error state (shown as a dismissible banner), and finally resets the loading state. Image load failures are handled via `onError` callbacks (hiding the element or marking load complete). `localStorage` `JSON.parse` calls in `auth.js` and `linksStore.js` don't have try/catch — that's a known improvement area.

Q: Explain the component composition pattern used.

A: `ModalWrapper` is the base component that provides modal structure and behavior (Escape key, backdrop click). Specialized modals (`ConfirmModal`, `EditLinkModal`, etc.) wrap `ModalWrapper` and provide their own content, buttons, and handlers. This avoids duplicating modal boilerplate across 4 components.

13. Code-Specific Questions

Q: What does `cleanTitle()` do?

A: It takes a raw title (which may contain HTML tags from Shlink's auto-fetch), strips all tags via regex, collapses multiple spaces, replaces hyphens and pipes with en-dashes, and trims whitespace. Returns null for falsy input.

Q: What does `extractDomain()` do?

A: Parses a URL with `new URL()`, returns the hostname without `www.` prefix. Returns empty string for invalid URLs.

Q: What does `normalizeShortUrl()` do?

A: Ensures all short URLs use the correct base (`192.168.1.15:8080`). It parses the URL, extracts the pathname, and prepends the configured `SHORT_BASE`. This handles cases where Shlink might return a different hostname.

Q: How does the filtered array work?

A: It's a `useMemo` that chains `.filter()` (search matching) and `.sort()` (by date). Dependencies are `[links, search, sort]`. The search matches against both `title` and `long` fields case-insensitively. Sort compares `Date` objects from the `date` field.

14. Quick Definitions

Term	Definition
SPA	Single Page Application — one HTML page, JS dynamically updates content
Zustand	Small state management library for React (~1KB)
bcrypt	Password hashing algorithm with built-in salt
localStorage	Browser API for persistent key-value storage (~5-10MB limit)
Shlink	Self-hosted URL shortener API (PHP/Slim, Docker)
useMemo	React hook that caches computed values between renders
useCallback	React hook that caches function references between renders
Controlled component	Form input whose value is controlled by React state
Prop drilling	Passing props through multiple component layers
Persist middleware	Zustand plugin that syncs store state to localStorage